

Gradient Boosting

M514: Econometrics II

Sangmyung Ha

Indiana University

Spring 2024

Outline

Gradient Descent

Functional Gradient Descent

Gradient Tree Boosting

Regression

Classification

Gradient Boosting: Implementation

References

1. Trevor Hastie, Robert Tibshirani, Jerome Friedman (2009), "The Elements of Statistical Learning (ESL)", Chapter 10.9 ~ 10.13
2. Tianqi Chen, Carlos Guestrin (2016), "XGBoost: A Scalable Tree Boosting System"
3. S. Boyd and L. Vandenberghe (2004), "Convex optimization", Chapter 9.1 ~ 9.4
4. Jerome Friedman (2001), Greedy Function Approximation: A Gradient Boosting Machine, The Annals of Statistics
5. Peter Buhlmann, Bin Yu (2003), Boosting with the L2 Loss: Regression and Classification

Steepest Descent Direction

Suppose $f : \mathbb{R}^p \rightarrow \mathbb{R}$ is differentiable at x_0 . Then, first order approximation of f at x_0 is given by

$$f(x) - f(x_0) \approx \nabla f(x_0)'(x - x_0)$$

Note that $f(x)$ increases fastest if we go from x_0 in the direction of the gradient of f : $\nabla f(x_0)$. To see this, note that from Cauchy-Schwarz inequality we have for any $u, v \in \mathbb{R}^p$,

$$u'v \leq \sqrt{u'u}\sqrt{v'v}$$

where the equality is obtained if $u = cv$ for some $c \in \mathbb{R}$.

Steepest Descent Direction

Take $u = \nabla f(x_0)$ and $v = x - x_0$. Then, the first order approximation

$$f(x) - f(x_0) \approx \nabla f(x_0)'(x - x_0)$$

is maximized if we take $x - x_0 = c \nabla f(x_0)$ for some $c \in \mathbb{R}$. This means that starting at x_0 , we need to move in the direction of $\nabla f(x_0)$ in order to maximize $f(x)$. Note that this is only a local argument because the first order approximation only holds for x around x_0 . Using the same argument, we may show that the negative gradient $-\nabla f(x_0)$ is the steepest descent direction.

Gradient Descent Method

Consider the unconstrained optimization problem below.

$$\min_{x \in \mathbb{R}^p} f(x)$$

We may find the solution to the problem by the **gradient descent** algorithm.

Gradient Descent

1. **Choose** a starting point $x \in \mathbb{R}^p$.
 2. **Repeat**
 - (a) Evaluate the steepest descent direction at x : $\Delta x = -\nabla f(x)$.
 - (b) Line Search: Choose step size t via exact (or backtracking) line search
 - (c) Update: $x \leftarrow x + t\Delta x$
 3. **until** stopping criterion is satisfied.
-

The stopping criterion is usually of the form $\|\nabla f(x)\|_2 \leq \epsilon$.

Gradient Descent Method

In **exact line search**, t is chosen to minimize f along the ray $\{x + t\Delta x | t \geq 0\}$:

$$t = \arg \min_{s \geq 0} f(x + s\Delta x)$$

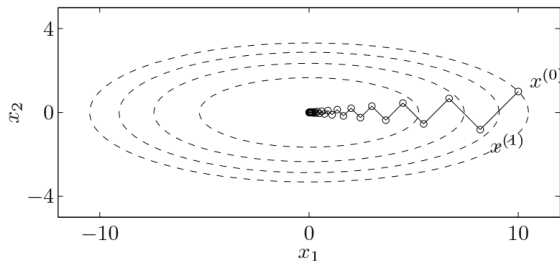


Figure 9.2 Some contour lines of the function $f(x) = (1/2)(x_1^2 + 10x_2^2)$. The condition number of the sublevel sets, which are ellipsoids, is exactly 10. The figure shows the iterates of the gradient method with exact line search, started at $x^{(0)} = (10, 1)$.

Figure 1: from: S. Boyd and L. Vandenberghe (2004) ,p 469

Functional Gradient Descent

Recall that AdaBoost can be interpreted as fitting the loss function:

$$\min_{\{\beta_m, G_m\}_{m=1}^M} \sum_{i=1}^n L(y_i, f(x_i)) \quad (1)$$

where $f(x_i) = \sum_{m=1}^M \beta_m G_m(x_i)$ using forward stagewise modeling and exponential loss function.

$$L(y, f(x)) = \exp \left(-y \left(\sum_{m=1}^M \beta_m G_m(x) \right) \right)$$

In other words, for $m = 1, \dots, M$, AdaBoost tries to solve

$$(\beta_m, G_m) = \arg \min_{\beta, G} \sum_{i=1}^n \exp(-y_i(f_{m-1}(x_i) + \beta G(x_i)))$$

at each iteration.

Functional Gradient Descent

- ▶ When using exponential loss function, forward stagewise modeling yields simple and fast algorithm (AdaBoost or SAMME). However, this is not true when using other loss functions such as binomial deviance loss or absolute deviation (Hastie et al., 2009, p357).
- ▶ In this case, fast approximate algorithms for solving (1) with any differentiable loss criterion can be derived using gradient descent algorithm.

Functional Gradient Descent

$$(\beta_m, G_m) = \arg \min_{\beta, G} \sum_{i=1}^n \exp(-y_i(f_{m-1}(x_i) + \beta G(x_i)))$$

The forward stagewise modeling in AdaBoost Algorithm is analogous to the gradient descent in numerical optimization. Given the weighted sum of previous fitted models $f_{m-1}(x)$, the updating step

$$f_m(x) = f_{m-1}(x) + \beta_m G_m(x)$$

can be viewed as the taking a steepest descent step (best greedy step) toward the optimal value $L^*(f) = \sum_{i=1}^n L(y_i, f(x_i))$, under the constraint that the step "direction" $G(x)$ be a classifier. Here, β can be interpreted as the step size.

Functional Gradient Descent

When the function f in (1) is not constrained to be a weighted sum of classifiers, solving (1) can be viewed as a numerical optimization problem:

$$\hat{\mathbf{f}} = \arg \min_{\mathbf{f}} L(\mathbf{f}) = \sum_{i=1}^n L(y_i, f(x_i)) \quad (2)$$

In this case, the input arguments $\mathbf{f} \in \mathbb{R}^n$ are the values of the function $f(x_i)$ at each of the samples $\{x_i\}_{i=1}^n$:

$$\mathbf{f} = (f(x_1), f(x_2), \dots, f(x_n))'$$

Iterative numerical optimization procedures solve (2) as a sum of component vectors

$$\mathbf{f}_M = \sum_{m=0}^M \mathbf{h}_m, \quad \mathbf{h} \in \mathbb{R}^N$$

where $\mathbf{f}_0 = \mathbf{h}_0$ is an initial guess, and each successive $\mathbf{f}_m$ is induced based on the current parameter vector $\mathbf{f}_{m-1}$, which is the sum of the previously induced updates.

Functional Gradient Descent

If minimizing loss on the training data were the only goal, i.e. finding $\mathbf{f}^*$, gradient descent would be the preferred strategy.

In gradient descent, we choose $\mathbf{h}_m = -t_m g_m$ where t_m is a scalar and $g_m \in \mathbb{R}^n$ is the gradient of $L(\mathbf{f})$ evaluated at $\mathbf{f} = \mathbf{f}_{m-1}$.

The components of g_m are

$$g_{im} = \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f(x_i)=f_{m-1}(x_i)}, \quad i = 1, \dots, n$$

The step size t_m is found by exact line search:

$$t_m = \arg \min_t L(\mathbf{f}_{m-1} - t g_m)$$

Using gradient descent, it is guaranteed that $L(\mathbf{f})$ will be minimized w.r.t $\mathbf{f}$ for large enough M , provided that $L(\mathbf{f})$ is convex.

Functional Gradient Descent

However, our goal is to generalize $f(x)$ to new data not represented in the training set. A possible resolution to this dilemma is to implement an algorithm which mimics gradient descent, but also allows us to evaluate the function $f(x)$ at any point in $x \in \mathbb{R}^p$.

This is done by **functional gradient descent**. In functional gradient descent, instead of taking a step in the steepest descent direction, a classifier which best approximates the negative gradient is found. Afterwards, the classifier is added to the existing model, with weights found by a method analogous to exact line search.

Functional Gradient Descent (Friedman, 2001), (Buhlmann, Yu, 2003)

Generic Functional Gradient Descent

1. (Initialization) Fit a learner

$$\hat{F}_0(x) = h(x; \hat{\theta}_{y,x})$$

When using least squares, $\hat{\theta}_{y,x} = \arg \min_{\theta} \sum_{i=1}^n (y_i - h(x_i; \theta))^2$

2. For $m = 1, \dots, M$

- (a) Compute the negative gradient vector,

$$r_i = - \left[\frac{\partial L(y_i, F)}{\partial F} \right]_{F=\hat{F}_{m-1}(x_i)}, \quad i = 1, \dots, n$$

evaluated at $\hat{F}_{m-1}(\cdot)$.

- (b) Fit the real-valued learner to the gradient vector,

$$\hat{f}_m(x) = h(x, \hat{\theta}_{r,x})$$

When using least squares, $\hat{\theta}_{r,x} = \arg \min_{\theta} \sum_{i=1}^n (r_i - h(x_i, \theta))^2$

- (c) Do one-dimensional numerical search for the best step-size

$$\hat{w}_m = \arg \min_w \sum_{i=1}^n L(y_i, \hat{F}_{m-1}(x_i) + w \hat{f}_m(x_i))$$

- (d) Update: $\hat{F}_m(\cdot) = \hat{F}_{m-1}(\cdot) + \hat{w}_m \hat{f}_m(\cdot)$

Gradient Tree Boosting (Hastie et al., 2009)

Hastie et al., (2009) presents the functional gradient descent for boosting decision trees for both regression and classification problems.

At each iteration, the algorithm is organized into two stages:

1. (Finding descent direction) Grow a tree which approximates the negative gradient. Use the partition of $\mathbb{R}^p$ induced by this tree for the next step.
2. (Line search) Instead of performing exact line search, find the constants corresponding to each partition found by Step 1 which minimizes the loss function.

Gradient Tree Boosting (Hastie et al., 2009)

Finding descent direction:

Induce a tree $T(x; \Theta_m)$ at the m -th iteration whose predictions are as close as possible to the negative gradient. Using squared error to measure closeness, this leads us to

$$\tilde{\Theta} = \arg \min_{\Theta} \sum_{i=1}^n (-g_{im} - T(x_i; \Theta))^2 \quad (3)$$

That is, one fits the tree T to the negative gradient values by least squares criterion.

Note that this step is induced only as a way to approximate the negative gradient. In gradient descent, negative gradient is used only to find the descent direction. Afterwards, line search is performed to determine how large a step to take. Likewise, the approximating tree is found to determine the partitions in decision trees only.

Gradient Tree Boosting (Hastie et al., 2009)

Line search:

After constructing the tree, the corresponding constants in each region are given by solving:

$$\hat{\gamma}_{jm} = \arg \min_{\gamma_{jm}} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma_{jm}) \quad (4)$$

for $j = 1, \dots, J_m$, where the index j corresponds to the terminal region. The solution to (4) is analogous to the exact line search in gradient descent. The difference is that (4) performs a separate line search for each separate terminal region $\{T(x_i; \Theta_m)\}_{x_i \in R_{jm}}$, $j = 1, \dots, J_m$.

Gradient Tree Boosting (Hastie et al., 2009, p361)

Gradient Boosting (Regression)

1. Initialize $f_0(x) = \arg \min_{\gamma} \sum_{i=1}^n L(y_i, \gamma)$.
2. For $m = 1, \dots, M$:
 - (a) For $i = 1, \dots, n$ compute

$$r_{im} = - \left[\frac{\partial L(y_i, f(x_i))}{\partial f(x_i)} \right]_{f=f_{m-1}}$$

- (b) Fit a regression tree to the targets r_{im} giving terminal regions $R_{jm}, j = 1, \dots, J_m$.
- (c) For $j = 1, \dots, J_m$ compute

$$\gamma_{jm} = \arg \min_{\gamma} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma)$$

- (d) Update $f_m(x) = f_{m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jm} 1_{\{x \in R_{jm}\}}$
3. Output $\hat{f}(x) = f_M(x)$.
-

Gradient Tree Boosting (Hastie et al., 2009)

Previous algorithm presents the generic gradient tree-boosting algorithm for regression. Specific algorithms are obtained by inserting different loss criteria $L(y, f(x))$. The first line of the algorithm initializes to the optimal constant model, which is just a single terminal node tree. The components of the negative gradient computed at line 2(a) are referred to as generalized or pseudo residuals: r . Gradients for commonly used loss functions are summarized below.

Gradient Tree Boosting (Hastie et al., 2009)

TABLE 10.2. *Gradients for commonly used loss functions.*

Setting	Loss Function	$-\partial L(y_i, f(x_i))/\partial f(x_i)$
Regression	$\frac{1}{2}[y_i - f(x_i)]^2$	$y_i - f(x_i)$
Regression	$ y_i - f(x_i) $	$\text{sign}[y_i - f(x_i)]$
Regression	Huber	$y_i - f(x_i)$ for $ y_i - f(x_i) \leq \delta_m$ $\delta_m \text{sign}[y_i - f(x_i)]$ for $ y_i - f(x_i) > \delta_m$ where $\delta_m = \alpha \text{th-quantile}\{ y_i - f(x_i) \}$
Classification	Deviance	k th component: $I(y_i = \mathcal{G}_k) - p_k(x_i)$

Figure 2: ESLII (p360)

Gradient Tree Boosting (Hastie et al., 2009)

classification

With K -class classification, the response Y takes values in the unordered set $\mathcal{G} = \{\mathcal{G}_1, \dots, \mathcal{G}_K\}$. We now seek a classifier $G(x)$ taking values in $\mathcal{G}$. We model the probabilities as

$$p_k(x) = \frac{e^{f_k(x)}}{\sum_{l=1}^K e^{f_l(x)}} \quad (5)$$

This leads to multinomial deviance loss function (or negative of the multinomial log likelihood):

$$\begin{aligned} L(y, p(x)) &= - \sum_{k=1}^K 1_{\{y=\mathcal{G}_k\}} \log p_k(x) \\ &= - \sum_{k=1}^K 1_{\{y=\mathcal{G}_k\}} f_k(x) + \log \left(\sum_{l=1}^K e^{f_l(x)} \right) \end{aligned} \quad (6)$$

Gradient Tree Boosting (Hastie et al., 2009)

classification

The algorithm for classification is similar. Lines 2(a)–2(d) of Gradient Boosting Algorithm are repeated K times at each iteration m , once for each class using:

$$\begin{aligned} -g_{ikm} &= \frac{\partial L(y_i, f_{1m}(x_i), \dots, f_{Km}(x_i))}{\partial f_{km}(x_i)} \\ &= 1_{\{y_i = \mathcal{G}_k\}} - p_k(x_i) \end{aligned} \tag{7}$$

The result at line 3 is K different (coupled) tree expansions $f_{kM}(x)$, $k = 1, \dots, K$. These produce probabilities via (5) or do classification as:

$$G(x) = \mathcal{G}_k \quad \text{where} \quad k = \arg \max_l p_l(x)$$

Gradient Tree Boosting (Hastie et al., 2009, p387)

classification

Gradient Boosting (Classification)

1. Initialize $f_{k0}(x) = 0$, $k = 1, \dots, K$.

2. For $m = 1, \dots, M$:

(a) Set

$$p_k(x) = \frac{e^{f_k(x)}}{\sum_{l=1}^K e^{f_l(x)}}$$

(b) For $k = 1$ to K :

i Compute $r_{ikm} = y_{ik} - p_k(x_i)$, $i = 1, 2, \dots, n$

ii Fit a regression tree to the targets r_{ikm} , $i = 1, 2, \dots, n$, giving terminal regions R_{jkm} , $j = 1, 2, \dots, J_m$.

iii Compute

$$\gamma_{jkm} = \frac{K-1}{K} \frac{\sum_{x_i \in R_{jkm}} r_{ikm}}{\sum_{x_i \in R_{jkm}} |r_{ikm}|(1 - |r_{ikm}|)}$$

iv Update $f_{km}(x) = f_{k,m-1}(x) + \sum_{j=1}^{J_m} \gamma_{jkm} I(x \in R_{jkm})$

3. Output $\hat{f}_k(x) = f_{kM}(x)$, $k = 1, \dots, K$.

Here, $y_{ik} = 1_{y_i=k}$.

Gradient Tree Boosting (Hastie et al., 2009, p387)

classification

In the Line 2.(b)iii of the previous algorithm,

$$\gamma_{jkm} = \frac{K-1}{K} \frac{\sum_{x_i \in R_{jkm}} r_{ikm}}{\sum_{x_i \in R_{jkm}} |r_{ikm}|(1 - |r_{ikm}|)}$$

is derived as the minimizer of the second order Taylor expansion of the loss (or negative log-likelihood) evaluated at the previous iterate. Minimizer of the exact loss function does not have a closed form solution and just one Newton step is taken as an approximation. Using the minimizer of the second order Taylor expansion can be seen as taking one Newton step in the Newton's method. For details, see Ex 10.8 and Ex 10.9 of the ESLii.

Implementation Details (based on XGBoost)

1. Learning rate (η)
2. Tree size (`max_depth`)
3. Penalization (λ, α)
4. Subsample (`subsample`), number of trees in each iteration (`num_parallel_tree`)

Learning rate (η)

To avoid overfitting, trees can be added into the model after shrinkage. That is, we update

$$f_m(x) = f_{m-1}(x) + \eta \sum_{j=1}^{J_m} \gamma_{jm} 1_{\{x \in R_{jm}\}}$$

where η is a value less than 1. The parameter η can be regarded as controlling the learning rate of the boosting procedure and has default value of 0.3.

Tree size (max_depth)

Unlike in Random Forest where people typically use ensemble of fully grown trees, in Gradient Boosting, people typically limit the maximum depth of the trees. Maximum depth is the length of the longest path from the tree root to a terminal node and controls the complexity of each tree.

In Random Forest, adding more trees does not cause overfitting. In Gradient Boosting, adding more trees causes overfitting.

Penalization (λ, α)

In the loss minimization step,

$$\min_{\{\gamma_{jm}\}_{j=1}^{J_m}} \sum_{j=1}^{J_m} \sum_{x_i \in R_{jm}} L(y_i, f_{m-1}(x_i) + \gamma) \quad (8)$$

we may add either l_1 or l_2 penalty term on the size of γ , i.e.,

$$\sum_{j=1}^{J_m} \gamma_{jm}^2 \quad \text{or} \quad \sum_{j=1}^{J_m} |\gamma_{jm}|$$

Suppose (8) has a closed form solution. Would the penalized loss function also have a closed form solution?

Subsample (subsample), number of trees in each iteration (num_parallel_tree)

Instead of boosting trees, we may boost a forest of trees. Here, forest of trees are grown using subsample (sampling without replacement) of the original trees.

- ▶ subsample: Ratio of samples to randomly draw in growing each trees
- ▶ num_parallel_tree: How many trees to grow in each iteration (each gradient descent step)

Documentation

For more, read the documentation for XGBoost:

<https://xgboost.readthedocs.io/en/stable/parameter.html>